

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Buenos Aires

**Trabajo Práctico N°1: Autómatas Finitos y
Análisis Léxico**

Materia: Sintaxis y Semántica de los Lenguajes

Año Lectivo: 2026

Docente: Ing. Roxana Leituz

Integrantes Grupo 24:

Nombre y Apellido	Legajo
Alan Maximo Ramirez	233.188-3
Gonzalo Agustin Ferreño	233.869-5
Lucas Panzera	234.084-7

1. Introducción

En este documento abarcamos el diseño, modelado formal e implementación de los autómatas finitos determinísticos (AFD) aplicados a los tres ejercicios planteados, explicando metodologías y decisiones a la hora del desarrollo.

Como metodología general de desarrollo, se eligió un diseño modular e incremental. Cada ejercicio se estructuró separando la fase de reconocimiento (mediante matrices de transición) de la fase de procesamiento (cálculo).

2. Ejercicio 1: Analizador Léxico de Constantes (Decimal, Octal, Hexadecimal)

2.1. Metodología y Análisis del Problema

El objetivo principal de esta parte es procesar una cadena que contiene múltiples constantes enteras (decimales, octales o hexadecimales, permitiendo signo opcional únicamente en las decimales) separadas por el carácter @. El programa debe clasificar cada constante, reportar errores léxicos en caso de existir, y contabilizar de forma independiente la cantidad de elementos correctos hallados en cada categoría.

2.2. Decisiones de Diseño

- **Modelado del AFD:** Se diseñó un autómata finito determinista de 9 estados. La estructura tiene un estado inicial que toma un flujo según el signo o la presencia de prefijos característicos (como el 0 inicial para octales/hexadecimales, o el signo en los decimales).
- **Mapeo de Entradas (columna):** Se implementó una función de abstracción que traduce caracteres individuales del alfabeto a un índice numérico de columna (agrupando signos, ceros, dígitos octales, dígitos decimales, letras hexadecimales y el separador @).
- **Conteo:** Se diseñó la lógica de conteo para que evalúe el estado de origen al momento de consumir el carácter separador @, asegurando que el incremento de los contadores (cant_dec, cant_oct, cant_hex) ocurra únicamente cuando la constante ha finalizado de manera exitosa.
- **Signo exclusivo de las decimales:** el estado 7 representa las constantes "+0" y "-0", y desde él solo se admite el separador. De esta manera "+017" y "+0x1F" resultan errores léxicos, ya que un 0 inicial convierte al número en octal o hexadecimal y en esos grupos el signo no está permitido.
- **Clasificación del 0:** una constante formada solo por el dígito 0 (estado 2), y las constantes "+0" y "-0" (estado 7), se cuentan como decimales. El grupo octal exige al menos un dígito después del 0 inicial, como en "00" o "017".

2.3. Definición Formal del Autómata

Significado de los estados: 0 inicio de constante, 1 signo leído, 2 cero inicial, 3 prefijo 0x, 4 decimal en curso, 5 octal en curso, 6 hexadecimal en curso, 7 cero con signo, 8 error.

3. Ejercicio 2: Conversión de Carácter a Entero

3.1. Metodología y Análisis del Problema

Este ejercicio resuelve la necesidad de transformar un carácter que representa un dígito en su respectivo valor numérico entero.

3.2. Decisiones de Diseño

- **Aprovechamiento de la Tabla ASCII:** En lugar de utilizar estructuras condicionales o funciones de bibliotecas externas, se diseñó la función basándose en la correlatividad numérica de los caracteres en la tabla ASCII ('0' al '9'). Al restar el valor ASCII del carácter '0' al carácter recibido (c - '0'), se obtiene directamente el valor entero equivalente de manera sumamente eficiente.
- **Control de Errores:** Se estableció que, si el carácter ingresado no pertenece al rango de dígitos válidos, la función debe retornar estrictamente el valor -1. Esto proporciona un mecanismo de control que luego aprovechamos en el flujo principal para reportar errores sin interrumpir la ejecución del programa.

4. Ejercicio 3: Validador de Expresiones Aritméticas y Resolución por Precedencia

4.1. Metodología y Análisis del Problema

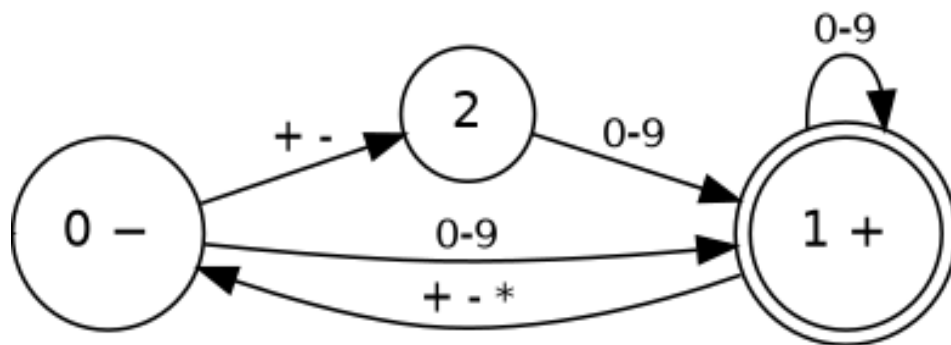
Este último ejercicio consta de dos etapas:

1. **Validación Sintáctica:** Tomar una cadena que representa una operación aritmética con enteros decimales (involucrando sumas, restas y multiplicaciones) y validarla mediante un AFD antes de hacer cualquier tipo de operación matemática.
2. **Evaluación de Precedencia:** Si la cadena es aceptada por el autómata (es decir, pertenece al lenguaje formal definido), se debe calcular su resultado matemático respetando la jerarquía de operadores (la multiplicación * tiene prioridad sobre la suma + y la resta -).

4.2. Definición Formal del Autómata

El AFD implementado se formaliza mediante la siguiente 5-tupla:

- $M = (Q, \Sigma, \delta, q_0, F)$
- **Conjunto de Estados (Q):** {0, 1, 2, 3}, donde el estado 3 actúa como estado de rechazo o trampa ante cualquier desvío sintáctico.
- **Alfabeto (Σ):** {0, 1-9, +, -, *}}
- **Estado Inicial (q_0):** 0
- **Estados de Aceptación (F):** {1} (la operación es válida únicamente si la cadena termina después de un número, sin el símbolo =).



No se graficó el estado de rechazo por prolijidad.

Estado	0	1-9	+ -	*	Otro
0 -	1	1	2	3	3
1 +	1	1	0	0	3
2	1	1	3	3	3
3	3	3	3	3	3

Significado de los estados: 0 se espera un número, al comienzo de la cadena o después de un operador; 1 número en curso; 2 signo leído; 3 error.

El alfabeto quedó limitado a los dígitos y a los operadores + - *, tal como los define la consigna, por lo que la operación no lleva ningún símbolo de cierre: "3+450*7+3-5" es una cadena válida y "3+450*7+3-5=" es un error léxico.

Se admite un signo delante de cualquier número, tanto al comienzo de la cadena como después de un operador (transiciones al estado 2). De esta manera "3+-4" se interpreta como 3 + (-4) y da -1, y "3*-4" da -12, el signo pertenece al número y no al operador. Solo se admite un signo, de modo que "3+--4" es un error léxico.

4.3. Decisiones de Diseño para la Evaluación de Precedencia

Para resolver el cálculo respetando la precedencia de operadores sin incorporar librerías externas complejas se realizó un procedimiento basado en dos pasadas sobre arreglos paralelos:

- **Extracción:** Se recorre la cadena validada reutilizando la función del Ejercicio 2 (char_a_entero). Esto permite agrupar de manera limpia números enteros de múltiples cifras multiplicando varias veces por 10, mientras se separan los operandos en un arreglo y los operadores en otro.
- **Primera Pasada:** Se realiza un recorrido exclusivo buscando los operadores *. Al detectarlos, se operan los números adyacentes. De esta forma, se resuelven las multiplicaciones en primer lugar.
- **Segunda Pasada:** Una vez eliminadas las multiplicaciones, se evalúan de izquierda a derecha las sumas y restas para dar el resultado final.

5. Capturas de Pantalla

```
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
$ ./main.exe

Cadena a analizar: +123@075@0xFF@0
Cantidad de Decimales: 2 | Cantidad de Octales: 1 | Cantidad de Hexa: 1

La cadena pertenece al lenguaje y fue procesada correctamente.

Cadena a analizar: 3+450*7+3-5

La cadena pertenece al lenguaje y fue procesada correctamente.

El resultado de la cuenta es: 3151
```

La primera cadena es correcta, tiene 2 decimales, 1 octal y 1 hexadecimal.

La segunda cadena también es válida, da como resultado 3151.

```
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
$ ./main.exe

Cadena a analizar: 45@017@0xFF@0
Cantidad de Decimales: 2 | Cantidad de Octales: 1 | Cantidad de Hexa: 1

La cadena pertenece al lenguaje y fue procesada correctamente.

Cadena a analizar: 10-2*3

La cadena pertenece al lenguaje y fue procesada correctamente.

El resultado de la cuenta es: 4
```

La primera cadena es válida, tiene 2 decimales, 1 octal y 1 hexadecimal.

La segunda cadena también es válida, da como resultado 4.

```
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
$ ./main.exe

Cadena a analizar: 0xABF@-152@0Xafe@015@5
Cantidad de Decimales: 2 | Cantidad de Octales: 1 | Cantidad de Hexa: 2

La cadena pertenece al lenguaje y fue procesada correctamente.

Cadena a analizar: 2+3*4*5

La cadena pertenece al lenguaje y fue procesada correctamente.

El resultado de la cuenta es: 62
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
```

La primera cadena es válida, tiene 2 decimales, 1 octal y 2 hexadecimales.

La segunda cadena también es válida, da como resultado 62.

```
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
$ ./main.exe

Cadena a analizar: 08@5

La cadena NO es valida (contiene errores lexicos).

Cadena a analizar: 3+

La cadena NO es valida (contiene errores lexicos).
```

La primera cadena no es válida, tiene un error léxico porque 8 no es un dígito octal.

La segunda cadena tampoco es válida, tiene un error léxico porque la cadena no puede terminar en un operador.

```
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
● $ ./main.exe

Cadena a analizar: 5@@5

La cadena NO es valida (contiene errores lexicos).

Cadena a analizar: 3+-4

La cadena pertenece al lenguaje y fue procesada correctamente.

El resultado de la cuenta es: -1
```

La primera cadena no es válida, tiene un error léxico porque tiene una constante vacía entre operadores.

La segunda cadena es válida, da como resultado -1 porque se lee como $3 + (-4)$

```
gonza@Gonza MINGW64 ~/Documents/Sintaxis/TP1/Final
● $ ./main.exe

Cadena a analizar: 0x@1

La cadena NO es valida (contiene errores lexicos).

Cadena a analizar: 3+--4

La cadena NO es valida (contiene errores lexicos).
```

La primera cadena no es válida, tiene un error léxico porque le falta el dígito hexadecimal después del prefijo.

La segunda cadena tampoco es válida, tiene un error léxico porque no se admiten dos signos seguidos.